DELISP: Efficient Discovery of Generalized Sequential Patterns by Delimited Pattern-Growth Technology

Ming-Yen Lin, Suh-Yin Lee*, and Sheng-Shun Wang
Department of Computer Science and Information Engineering
National Chiao Tung University, Taiwan 30050, R.O.C.
Email: {mylin, sylee, sswang}@csie.nctu.edu.tw

Abstract

An active research in data mining is the discovery of sequential patterns, which finds all frequent sub-sequences in a sequence database. Most of the studies specify no time constraints such as maximum/minimum gaps between adjacent elements of a pattern in the mining so that the resultant patterns may be uninteresting. In addition, a data sequence containing a pattern is rigidly defined as only when each element of the pattern is contained in a distinct element of the sequence. This limitation might lose useful patterns for some applications because sometimes items of an element might be spread across adjoining elements within a specified time period or time window. Therefore, we propose a pattern-growth approach for mining the generalized sequential patterns. Our approach features in reducing the size of sub-databases by bounded and windowed projection techniques. Bounded projections keep only time-gap valid sub-sequences and windowed projections save non-redundant sub-sequences satisfying the sliding time window constraint. Furthermore, the delimited growth technique directly generates constraint-satisfactory patterns and speeds up the growing process. The empirical evaluations show that the proposed approach has good linear scalability and outperforms the well-known *GSP* algorithm in the discovery of generalized sequential patterns.

Keywords: data mining, sequence mining, generalized sequential pattern, time constraint.

1. Introduction

Sequential pattern mining is one of the active researches in data mining [1, 2, 3, 7, 8, 12]. The focus has been on improving the efficiency of the mining process owing to the huge number of potential patterns [1, 4, 14]. Typically, a user specifies a minimum support threshold to locate all the frequent sub-sequences, called *sequential patterns*, from a set of data sequences. Nevertheless, the time constraints between elements of a sequential pattern are not specified so that some uninteresting patterns may appear. For example, without specifying the maximum time gap, one may find a pattern <(*b*, *d*, *e*)(*a*, *f*)>, which means an item-set having *a* and *f* will occur after the occurrence of an item-set having *b*, *d*, and *e*. However, the pattern could be insignificant if the time interval between the two item-sets is too long such as over months. Therefore, time constraints including *maximum gap* and *minimum gap* should be incorporated in the mining to reinforce the accuracy and significance of mining results.

Moreover, common definition of an element of a sequential pattern is too rigid for some applications. Essentially, a data sequence is defined to support a pattern if each element of the pattern is contained in an individual element of the sequence, in the occurrence order of elements. However, a user may not care whether the items in an element (of the pattern) are from a single element or adjoining elements of a data sequence if the adjoining elements appear within a specified time interval. The specified interval is named *sliding* (*time*) *window* [11]. For instance, given a sliding window of *7*, a data sequence $<_{t1}(d, e)_{t2}(b)_{t3}(a, f)>$ can support the pattern $<(b, d, e)(a, f)>$ if the difference between time *t1* and time *t2* is no greater than *7*. Relaxing the definition of an element with sliding window will broaden the applications of sequential patterns.

Many algorithms were proposed for sequence mining without time constraints [1, 3, 6, 7, 12]. The Apriori-like algorithms [1, 5, 10] iteratively generate candidate patterns and determine true ones in multiple database scans, employing the Apriori heuristic (*all sub-patterns of a frequent pattern must be frequent*) in candidate pruning. The *SPADE* algorithm [14] quickly searches in a lattice constructed from the database of vertical-layout for the sequential patterns. The *FreeSpan* [4] projects sequences into sub-databases to confine the search and growth of sub-sequences. The *PrefixSpan* [9], representing a new methodology for the discovery of sequential patterns, explores prefix-projection and efficiently mines the complete set of sequential patterns. Nevertheless, these algorithms discover only patterns without the time constraints.

The *GSP* (**G**eneralized **S**equential **P**attern) algorithm, also an Apriori-like algorithm, was introduced to solve the problem of generalized sequential patterns [11]. In every database scan, each data sequence is transformed into time-lists of items for fast finding of certain element with a time tag. Since the start-time and end-time of a transaction (may comprise several elements) must be considered, *GSP* defined 'contiguous sub-sequence' for candidate generation, and moved between 'forward phase' and 'backward phase' for checking whether a data sequence contains a certain candidate [11].

The *cSPADE* [13] algorithm extended the *SPADE* [14] algorithm and modified the 'idlists' in *SPADE* to incorporate *min_gap*, *max_gap*, *time window*, and some other constraints. However, the time window refers to the window of occurrence of the whole sequence, rather than the sliding time window described here.

In this paper, we extend the pattern-growth technique [4] and propose a new algorithm, the *DELISP* (**Deli**mited **S**equential **P**attern) mining algorithm, for the discovery of sequential patterns generalized with constraints of maximum/minimum gap and sliding window. The *DELISP* grows sequential patterns in each sub-database generated by projecting each unfinished sub-sequence according to certain frequent prefixed elements. In *DELISP*, the *bounded projection* technique eliminates invalid sub-sequence projections caused by unqualified maximum/minimum gaps, the

windowed projection technique reduces redundant projections for adjacent elements satisfying the sliding window constraint, and the *delimited growth* technique grows only patterns satisfying constraints. The conducted experiments show that the *DELISP* outperforms the *GSP* algorithm. The scale-up experiments also confirm that the *DELISP* has good linear scalability with the number of data sequences.

The remaining paper is organized as follows. We formulate the problem and show an example in Section 2. Section 3 presents the *DELISP* algorithm. The experimental evaluation is described in Section 4. We discuss the performance factors of *DELISP* in Section 5. Section 6 concludes our study.

2. Problem Statement

Let $\Psi = \{\alpha_1, \alpha_2, \ldots, \alpha_z\}$ be a set of literals, called *items*. An *itemset* $I = (\beta_1, \beta_2, \ldots, \beta_m)$ is a set of m items, such that $I \subseteq \Psi$. A *sequence* x, denoted by $<a_1a_2\ldots a_n>$, is an ordered list of n *elements* where each *element* a_j is an itemset. Without loss of generality, we assume the items in an element are in lexicographic order. A sequence $\omega = <b_1b_2\ldots b_w>$ is a *subsequence* of another sequence $\varpi = <a_1a_2\ldots a_n>$ if there exist $1 \le i_1 < i_2 < \ldots < i_w \le n$ such that $b_1 \subseteq a_{i1}$, $b_2 \subseteq a_{i2}$, …, and $b_w \subseteq a_{iw}$.

A user specifies four parameters, *min_gap* (minimum gap), *max_gap* (maximum gap), *t_win* (sliding time window), and *minsup* (minimum support), for mining generalized sequential patterns in the database *DB* of data sequences. A *data sequence* is represented by $sid/<_{t1}a_{1\ t2}a_2\ldots{}_{tn}a_n>$, where element a_j occurred at time t_j, $t_1 < t_2 <\ldots< t_n$, and *sid* is a unique identifier. A data sequence $ds=sid/<_{t1}a_{1\ t2}a_2\ldots{}_{tn}a_n>$ *contains* a sequence $x = <b_1b_2\ldots b_w>$ if there exist integers $1 \le l_1 \le u_1 < l_2 \le u_2 < \ldots < l_w \le u_w \le n$ such that the four conditions hold: (1) $b_i \subseteq a_{l_i} \cup \ldots \cup a_{u_i}$, $1 \le i \le w$ (2) $t_{u_i} - t_{l_i} \le t_win$, $1 \le i \le w$ (3) $t_{u_i} - t_{l_{i-1}} \le max_gap$, $2 \le i \le w$ (4) $t_{l_i} - t_{u_{i-1}} > min_gap$, $2 \le i \le w$. Assume that t_j, *min_gap*, *max_gap*, and *t_win* are all positive integers and *min_gap* < *max_gap*.

The *minsup* is the user specified minimum support threshold. The *support* of a sequence x, denoted by *x.sup*, is the number of data sequences containing x divided by the total number of data sequences in *DB*. A sequence x is a *frequent sequence* if $x.sup \ge minsup$. The sequence x is also called a (**generalized**) ***sequential pattern***. Given the *minsup*, the three constraints *min_gap*, *max_gap*, *t_win*, and the database *DB*, the problem of generalized sequential pattern mining is to discover the set of all (generalized) sequential patterns. Note that the mining of sequential patterns without generalized constrains is a special case with *min_gap*=0, *max_gap*=∞, and *t_win*=0 here.

Example 1: Generalized sequential patterns in an example database

An example sequence database having 5 sequences is listed in the first column in Table 1. Take the first sequence for instance. It has two itemsets, one having a single item *c* occurring at time *1* and the other having items *b* and *f* occurring at time *35*. The sequential patterns with and without time

Table 1. Example sequence database *DB* and the sequential patterns

Sequence	**Sequential patterns (*minsup=40%*)**	**Generalized sequential patterns (*minsup=40%, min_gap=2, max_gap=30, t_win=2*)**
C1/<$_1$(c)$_{35}$(b,f)> *C2*/<$_2$(b)$_4$(d)> *C3*/<$_1$(a,d)$_5$(c)$_6$(c)$_8$(b)$_{35}$(a,f)> *C4*/<$_2$(a)$_4$(d)$_{30}$(f)$_{33}$(a)$_{61}$(f)> *C5*/<$_1$(a,b,e)$_4$(e)$_7$(f)$_8$(d)$_9$(b)>	<(a)>, <u><(a)(a)></u>, <(a)(b)>, <u><(a)(d)></u>, <(a)(f)>, <(b)>, <u><(b)(d)></u>, <(b)(f)>, <(c)>, <u><(c)(b)></u>, <u><(c)(f)></u>, <(d)>, <u><(d)(a)></u>, <u><(d)(b)></u>, <u><(d)(f)></u>, <(f)>	<(a)>,<(a)(b)>, <(*a,d*)>, <(a)(f)>, <(b)>, <(*b,d*)>, <(*b,f*)>, <(b)(f)>, <(c)>, <(d)>, <(f)>

constraints in the example database are illustrated as follows.

(i) Given the *minsup* = 40%. The <(a)>, occurring at *C3*, *C4*, and *C5*, is a sequential pattern. The <(a)(a)> (contained in *C3* and *C4*) and <(a)(d)> (contained in *C4* and *C5*) are both sequential patterns. The set of all sequential patterns are listed in the second column in Table 1.

(ii) Given the *minsup* = 40%, *min_gap* = 2, *max_gap* = 30, *t_win* = 2. The *C1* does not contain <(c)(b)> or <(c)(f)> now since the time gap (35-1) > *max_gap* (30). The *C2* does not contain <(b)(d)> any more since the time gap is not bigger than *min_gap*. However, <(a,d)> becomes a (generalized) sequential pattern because, with *t_win*=2, *C4* may contain <(a,d)> now. Likewise, *C5* may contain <(b,f)> due to *t_win*=2 so that <(b,f)> becomes a new pattern. The set of all generalized sequential patterns are listed in the third column in Table 1. We also underline patterns that turn out to be invalid and italicize the new sequential patterns, due to the specified constraints.

3. DELISP: Delimited Sequential Pattern Mining

In this section, we propose a new pattern-growth method for mining generalized sequential patterns, called *DELISP*. Analogous to the pattern-growth approaches [4, 9], the main idea is finding the frequent items, and then 'growing' potential patterns in the sub-databases constructed by projecting sub-sequences corresponding to the frequent items. However, *DELISP* projects fewer but complete combinations by windowed and bounded projections, and grows potential patterns effectively by delimited growth. In Section 3.1, we demonstrate the method by mining the example database in Table 1. Section 3.2 describes the proposed algorithm.

3.1 Mining generalized sequential patterns by *DELISP*: an example

Definition 1 (**Prefix**) Given a sequence $\varpi = <a_1 a_2 \ldots a_n>$, a sequence $\omega = <a_1' a_2' \ldots a_m'>$ ($m \le n$) is called a *prefix* of ϖ if and only if (1) $a_i' = a_i$, $1 \le i \le m-1$; (2) $a_m' \subseteq a_m$; and (3) for each item $\beta_j \in (a_m - a_m')$, $\beta_j > \beta_j'$, for all $\beta_j' \in a_m'$.

Definition 2 (**Stem-item, type-1 growth, and type-2 growth**) Given a sequential pattern ρ and a

frequent item x in the database *DB*, x is the *stem-item* of the sequential pattern ρ' if ρ' can be formed by (1) appending (x) as a new element to ρ or (2) extending the last element of ρ with x. The formation of ρ' is a *type-1 growth* if it is formed by appending (x), and a *type-2 growth* if it is formed by extending with x.

Example 2: Given *minsup*=40%, *min_gap*=2, *max_gap*=30, *t_win*=2, and the *DB* in Table 1. *DELISP* mines the patterns by the following steps.

Step 1. Find frequent items. By scanning *DB* once, we have <(a)>:3, <(b)>:4, <(c)>:2, <(d)>:4, <(e)>:1, and <(f)>:4. The notation *sequence_x:count* means that the support count of the *sequence_x* is *count*. Non-frequent item '*e*' is omitted from mining afterward.

Step 2. Project corresponding sub-sequences to sub-databases. Considering the generalized sequential patterns having ρ = <(x)> as the prefix, each can be found in the sub-database, named ρ-*DB*, generated by projecting all data sequences having item x in *DB*. Assume that ${}_{t_\rho}a_\rho$ is the first element containing the stem-item of ρ in a data sequence *ds*. We omit non-frequent items and project each element ${}_{t_j}a_j$ of *ds* into ρ-*DB* sequentially, according to the following rules.

(R0) $t_j < t_\rho$-*t_win*: the a_j is dropped.

(R1) (a) t_ρ-*t_win* $\le t_j \le t_\rho$: the items in a_j ordered after ρ are projected. (b) $t_\rho < t_j \le t_\rho$+*t_win*: project a_j.

(R2) t_ρ+*t_win* $< t_j$: let the first element in this range be ${}_{t_k}a_k$. The a_j having $t_j > t_\rho$+*min_gap* is projected if $t_k \le t_\rho$+*max_gap*.

Note that for each new data sequence, we mark t_ρ as the start-time (abbreviated as *st*) and the end-time (abbreviated as *et*) for use later. We tabulate the sub-databases <(a)>-*DB*, <(b)>-*DB*, <(c)>-*DB*, <(d)>-*DB*, and <(f)>-*DB* in part 1 of Table 2.

Table 2. The projected sub-sequences in the ρ-*DB* sub-databases

ρ-DB	**Projected sub-sequences**
part 1: sub-databases of *DB*	
<(a)>-*DB*	1:1/*C3*/<$_1$(d)$_5$(c)$_6$(c)$_8$(b)$_{35}$(a,f)>; 2:2/*C4*/<$_4$(d)$_{30}$(f)$_{33}$(a)$_{61}$(f)>; 1:1/*C5*/<$_1$(b)$_7$(f)$_8$(d)$_9$(b)>
<(b)>-*DB*	35:35/*C1*/<$_{35}$(f)>; 2:2/*C2*/<$_4$(d)>; 8:8/*C3*/<$_6$(c)$_{35}$(a,f)>; 1:1/*C5*/<$_7$(f)$_8$(d)$_9$(b)>
<(c)>-*DB*	5:5/*C3*/<$_8$(b)$_{35}$(a,f)>
<(d)>-*DB*	1:1/*C3*/<$_5$(c)$_6$(c)$_8$(b)$_{35}$(a,f)>; 4:4/*C4*/<$_{30}$(f)$_{33}$(a)$_{61}$(f)>; 8:8/*C5*/<$_7$(f)>
<(f)>-*DB*	30:30/*C4*/<$_{33}$(a)$_{61}$(f)>
part 2: sub-databases of <(a)>-*DB*	
<(a)(b)>-*DB*	8:8/*C3*/<$_{35}$(f)>
<(a,d)>-*DB*	1:1/*C3*/<$_8$(b)$_{35}$(f)>; 2:4/*C4*/<$_{30}$(f) $_{61}$(f)>
<(a)(f)>-*DB*	None
Note: the notation '*st*:*et*' prior to a data sequence denotes the *start-time* and the *end-time* of the data sequence with respect to ρ projection.	

Take <(a)>-*DB* for instance. For *C3*/<$_1$(**a**,d)$_5$(c)$_6$(c)$_8$(b)$_{35}$(a,f)>, we mark *st*:*et* as 1:1, and project the remaining item 'd' as the first element (rule/R1). The time of the next element is 5, hence we project all the remaining elements since the *min_gap*/*max_gap* constraints are satisfied (rule/R2). For

C4/<$_2$(**a**)$_4$(d)$_{30}$(f)$_{33}$(a)$_{61}$(f)>, the *st*:*et* is marked as 2:2, $_4$(d) is projected (rule/R1), and then all the remaining elements after time 2+min(2,2) are projected (rule/R2). Similar to the projection of *C3*, we have 1:1/*C5*/<$_1$(b)$_7$(f)$_8$(d)$_9$(b)> in <(a)>-*DB* from the projection of *C5*/<$_1$(**a**,b)$_4$(e)$_7$(f)$_8$(d)$_9$(b)>.

Note that while projecting *C3* to <(b)>-*DB*, *st*:*et*=8:8 and all the elements before time (8-2) are dropped (rule/R0). The projection completes after including $_6$(c) by rule/R1 and $_{35}$(a,f) by rule/R2. The rule/R1 also includes $_7$(f) in the projection of *C5* in <(d)>-*DB* (*st*:*et*=8:8). However, the $_9$(b) of *C5* is not projected into <(d)>-*DB* owing to b < d. In other words, the case that *C5* contains (b,d) with sliding-window is considered in <(b)>-*DB* instead of <(d)>-*DB*. Similarly, the $_9$(b) of *C5* in *DB* need not be kept in <(f)>-*DB* (*st*:*et*=7:7) for b < f. Moreover, the $_6$(c) in <(c)>-*DB* for the projection of *C3* (*st*:*et*=5:5) and the $_8$(d) in <(f)>-*DB* (*st*:*et*=7:7) for the projection of *C5* are dropped due to the unqualified *min_gap* constraint (rule/R2).

Step 3. Mine each sub-database for the subsets of generalized sequential patterns. In each sub-database, we grow the patterns in each sequence according to the time constraints, and determine which pattern is a valid sequential pattern. Assume that we are growing patterns from a sequence with *st*:*et* mark. The type-1 growth appends a new element with the item within the (*et*+*min_gap*, *st*+*max_gap*] range, and the type-2 puts the item within the [*et*-*t_win*, *st*+*t_win*] range in the same element having the ρ prefix. We mine <(a)>-*DB* as follows.

We grow the type-1 patterns <(a)(c)> and <(a)(b)> from the sub-sequence of *C3* using items from (1+2, 1+30]. In addition to the <(a,f)> at time 35, we also have pattern <(a,d)> from [1-2, 1+2]. The sub-sequence of *C4* generates <(a)(f)> from range (2+2, 2+30], and <(a,d)> from [2-2,2+2]. For the case of *C5*, we have <(a)(f)>, <(a)(d)>, and <(a)(b)> from the range (1+2, 1+30] and <(a,b)> from [1-2,1+2]. Consequently, the generalized sequential patterns are <(a)(b)>:2, <(a,d)>:2, and <(a)(f)>:2 by mining <(a)>-*DB*.

Step 4. Find all patterns by applying step 2 and step 3 on the sub-databases recursively. Next, we can apply the rules in step 2 to project the sub-sequences in <(a)>-*DB* further into sub-databases <(a)(b)>-*DB*, <(a,d)>-*DB*, and <(a)(f)>-*DB*. In step 2, the occurrence-time of the first element containing the stem-item determines how elements are projected. Assume that we are projecting a sequence with *st*:*et* mark. The occurrence-time of the searching element must be after *et* for sub-databases of type-1 growth (e.g. <(a)(b)>-*DB*), and within [*et*-*t_win*, *st*+*t_win*] for the other (e.g. <(a,d)>-*DB*). With $_{t_\rho}a_\rho$ being the first element found, we mark t_ρ:t_ρ as the *st*:*et* of the new sequence in type-1 growth. However, we use t_ρ to update either *st* or *et* of the new sequence of the other type. The projected sub-databases of <(a)>-*DB* are shown in part 2 of Table 2. We proceed to the mining of these sub-databases.

First on <(a)(b)>-*DB*, we mine the generalized sequential patterns having prefix (a)(b) by applying step 3. Since <(a)(b)(f)> has insufficient support, the mining stops. We then recursively

apply the above steps on <(a,d)>-*DB* and on <(a)(f)>-*DB*. Since no patterns are found so that the mining for patterns having prefix 'a' in the 'a' prefixed sub-databases completely stops. We then recursively apply the steps on <(b)>-*DB* for 'b' prefixed patterns, on <(c)>-*DB* for 'c' prefixed patterns, …, and on <(f)>-*DB* for 'f' prefixed patterns.

By collecting the patterns found in the above process, *DELISP* efficiently discovers all the sequential patterns satisfying the generalized constraints.

3.2 The *DELISP* Algorithm

Intuitively, one may modify the *PrefixSpan* Algorithm [4] to handle the *min_gap* and *max_gap* constraints. The modification requires recording the time-difference for each growing pattern, and invalidating patterns with insufficient/excess time gaps. However, the extra effort for verifying patterns will slow the whole process. In addition, modifying *PrefixSpan* algorithm to incorporate the sliding window constraint is a demanding task since *t_win* allows combining bi-directional elements. Growing patterns in either way would destroy the correctness of the pattern-growth framework.

Analogous to *PrefixSpan* algorithm, *DELISP* decomposes the mining problem by recursively growing patterns, one item longer than the current patterns, in the projected sub-databases. However, the potential items used to grow are subjected to *min_gap* and *max_gap* constraints, called *de-limited growth*. Therefore, we apply type-1 growth with item within range [et+*min_gap*, st+*max_gap*], and apply type-2 growth with item from range [et-*t_win*, st+*t_win*], where st and et denotes the time span for the current pattern. On projecting sub-databases, we avoid the bi-directional growth by imposing the item-order to type-2 growth. We always add a new item lexicographically ordered after all the items existing in the type-2 growth pattern. Considering an example element (b, d, e) formed by combining t_1(d, e) and t_2(b). When the time t_2 is earlier than time t_1, (b, d, e) will be discovered in the projected b-*DB* if (t_1-t_2) ≤ *t_win*. In case $t_1 < t_2$, we keep (d, e) in b-*DB* while projecting by allowing elements having time tag later than (t_2-*t_win*) be projected. We refer such projection to as *windowed-projection*. Fig. 1 presents the proposed *DELISP* algorithm.

4. Experimental Results

We conducted several experiments to access the performance of the *DELISP* algorithm. The experiments used an 866 MHz Pentium-III PC with 1024MB memory running the Windows NT. Like most studies on sequential pattern mining [1, 4, 11, 14], we generated the synthetic datasets for these experiments using the procedure described in [11]. Due to space limit, we only report the results on dataset C*10*–T*2.5*–S*4*–I*2.5* having 100,000 sequences and N=1000, N_S=5000, N_I=25000. We refer readers to [11] for the details of the parameters.

We compare the execution times of *GSP* with *DELISP* in mining generalized sequential patterns with various values of *minsup*, *min_gap*, *max_gap*, and *t_win*. In these experiments, *DELISP*

Algorithm *DELISP*

Input: *DB* = a sequence database; *minsup* = minimum support; *min_gap* = minimum time gap; *max_gap* = maximum time gap; *t_win* = sliding time window.

Output: the set of all generalized sequential patterns.

Method:

1. Scan *DB* once, find the set of all frequent items.
2. For each frequent item x,
   (i) form a sequential pattern $\rho = <(x)>$ and output ρ.
   (ii) call *ProjectDB*(ρ, <>-*DB*) to construct ρ-projected database ρ-*DB*.
   (iii) call *Mine*(ρ-*DB*).

Subroutine *ProjectDB*(ρ', ρ-*DB*)

Parameters: ρ' = a sequential pattern; ρ-*DB* = the sequence database *DB* if $\rho = <>$, or the ρ-projected database.

Output: the ρ'-projected database ρ'-*DB*.

Method:

1. If $\rho = <>$ then execute (1a), otherwise execute (1b).
   (1a) For each sequence $ds = sid/<_{t1}a_1\ _{t2}a_2 \ldots\ _{tn}a_n>$ in *DB*, find the first element $_{t\rho}a_\rho$ which contains x, where x is the stem-item of ρ'. Mark the *st*:*et* of a new sequence *ds'* as t_ρ:t_ρ.
   (1b) For each sequence $ds = st$:$et/sid/<_{t1}a_1\ _{t2}a_2 \ldots\ _{tn}a_n>$ in ρ-*DB*, find the first element $_{t\rho}a_\rho$ which contains x, where x is the stem-item of ρ' and
   (i) $et+min_gap < t_\rho \le st+max_gap$ if ρ' is a type-1 growth pattern. Mark the *st*:*et* of a new sequence *ds'* as t_ρ:t_ρ.
   (ii) $et-t_win \le t_\rho \le st+t_win$ if ρ' is a type-2 growth pattern. Mark the *start-time*:*end-time* of a new sequence *ds'* as $\min(st, t_\rho)$:$\max(et, t_\rho)$.
2. (**Windowed-projection**) For each element a_j such that $t_\rho - t_win \le t_j \le t_\rho$, insert $_{tj}a_j'$ to *ds'*, where a_j' is the set of all frequent items α_i in a_j satisfying $\alpha_i > x$. For each a_j such that $t_\rho < t_j \le t_\rho + t_win$, insert $_{tj}a_j$ to *ds'*.
3. (**Bounded-projection**) Find the first a_k such that $t_k > t_\rho + \max(t_win, min_gap)$. Insert $_{tk}a_k$, $_{tk+1}a_{k+1}$, …, and $_{tn}a_n$ to *ds'* if $t_k \le t_\rho + max_gap$.
4. Insert *ds'* to ρ'-*DB* if *ds'* is not empty.

Subroutine *Mine*(ρ-*DB*)

Parameter: ρ-*DB* = the ρ-projected database.

Method:

1. (**Delimited-growth**) For each data sequence $ds = st$:$et/sid/<_{t1}a_1\ _{t2}a_2 \ldots\ _{tn}a_n>$ in ρ-*DB*,
   (i) for each item x in a_i such that $et+min_gap < t_i \le st+max_gap$, increase the support count of potential type-1 growth pattern ρ' having the stem-item x.
   (ii) for each item x in a_i such that $et-t_win \le t_i \le st+t_win$, increase the support count of potential type-2 growth pattern ρ' having the stem-item x.
2. Validate the supports of the potential patterns in 1. Find the set of frequent items x which forms a type-1 or a type-2 growth pattern.
3. For each frequent item x,
   (i) form a sequential pattern ρ' and output ρ'.
   (ii) call *ProjectDB*(ρ', ρ-*DB*) to construct ρ'-projected database ρ'-*DB*.
   (iii) call *Mine*(ρ'-*DB*).

Fig. 1. Algorithm *DELISP*

is about 3 times faster than *GSP*. The result of varying *min_gap* is shown in Fig. 2 and Fig. 3 shows the result of varying *max_gap*. When we increase the *min_gap* or decrease the *max_gap* value, the number of generalized sequential patterns will decrease since both changes restrict some data

sequences to contain certain patterns. Fig. 4 displays the effect on performance when constraint *t_win* is increased. More patterns are discovered, and more execution time is required, with the increased *t_win*. The results of varying *minsup* (2%, 1.5%, 1%, 0.75%, 0.5%) are consistent. Finally, Fig. 5 shows that the execution time of *DELISP* scales up linearly with the number of customer sequences.

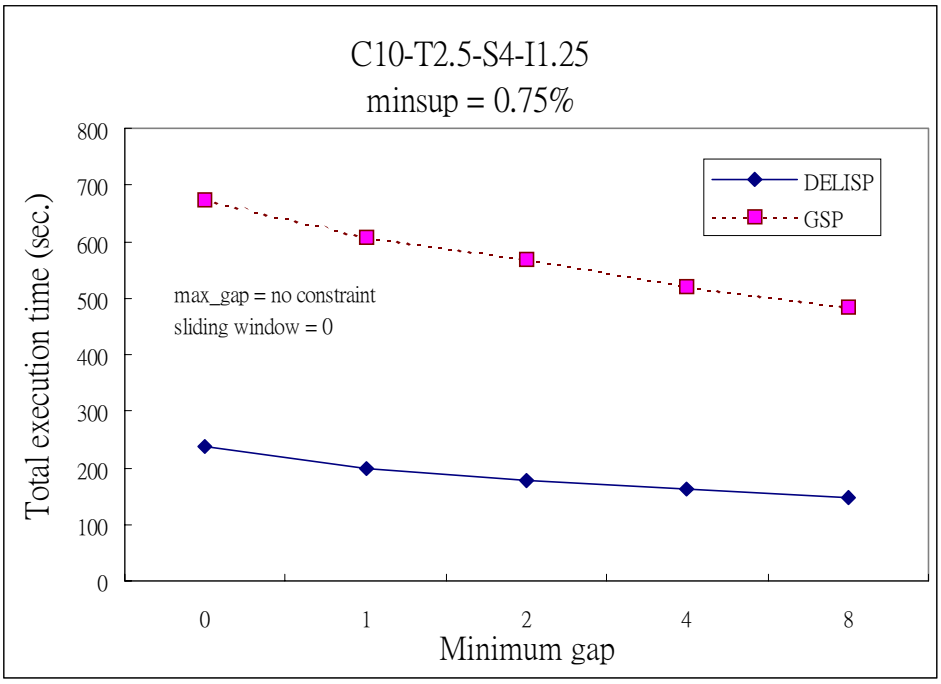


Fig.2. Effect of the *min_gap* constraint

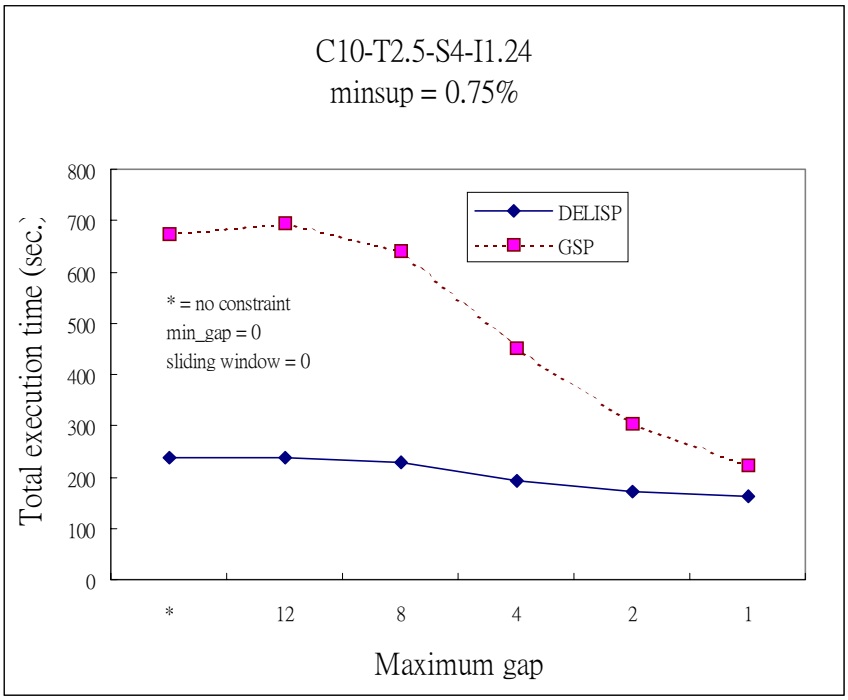


Fig. 3. Effect of the *max_gap* constraint

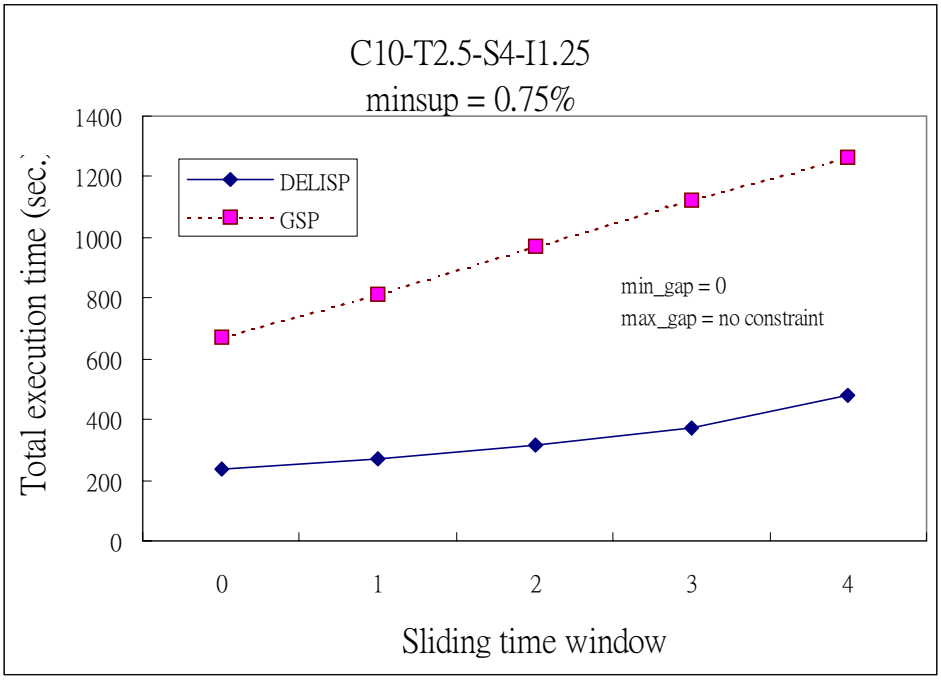


Fig. 4. Effect of the *t_win* constraint

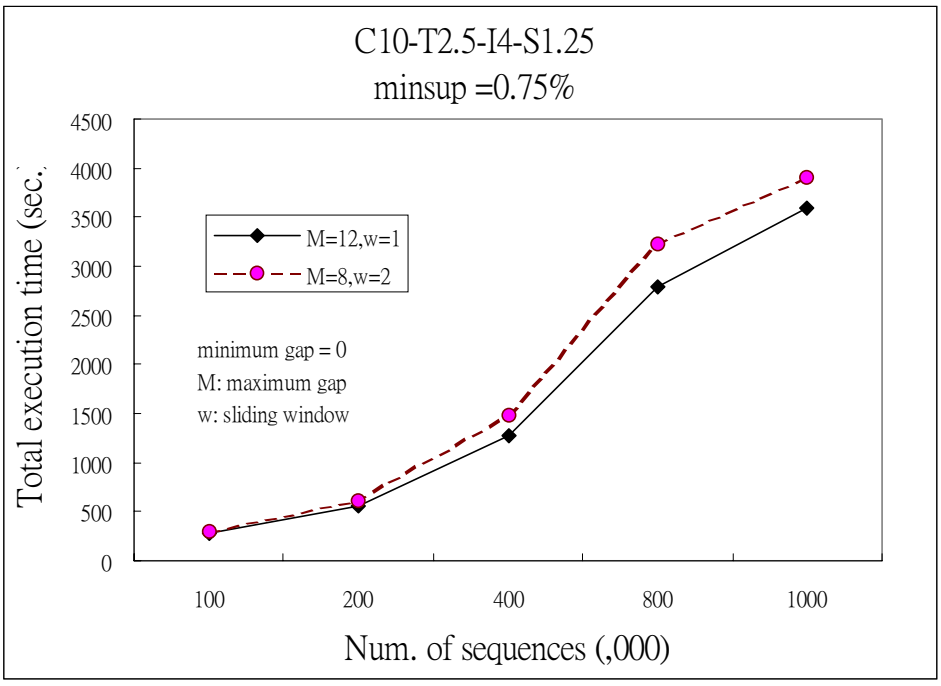


Fig. 5. Linear scalability of *DELISP*

5. Discussion

In this section, we summarize the factors contributing to the efficiency of the proposed *DELISP* algorithms, by comparison with the well-known *GSP* algorithm.

- **No candidate generation.** *DELISP* generates no candidates and saves the time for not only candidate generation but also candidate testing. Moreover, the huge space required for candidate

hash-tree is eliminated thoroughly. When we decreased the amount of installed memory (e.g. from 1024M down to 256M), the candidate *2*-sequences no longer fit into the memory at the same time. *GSP* needs to partition the candidates and scan the database more than once for support counting. *DELISP* grows patterns directly and frees from such limitation.

- **Focused search.** *DELISP* grows patterns by considering prefix-matched data sequences only. We search and grow longer patterns in the smaller but promising subspace. In contrast, *GSP* takes every data sequence for support calculation in each pass.

- **Constraint integration.** The *max_gap* constraint makes candidate reduction less powerful in *GSP* since some candidates cannot be pruned in advance. For instance, given *max_gap* constraint, a data sequence which supports candidate (a)(e)(f) may not contain candidate (a)(f). Thus, *GSP* must keep candidate (a)(e)(f) even if (a)(f) is rejected to be frequent by the *max_gap* constraint. Nevertheless, *DELISP* benefits from the constraint by incorporating the constraint in growing. Some postfix of a sequence may be out of consideration due to the *max_gap* constraint.

- **Containment checking and sequence shrinking.** In each pass, *GSP* transforms every data sequence into item-transaction-time lists, and switches between alternative phases with excess "pull up" of elements to check whether a data sequence contains a candidate [11]. Without any transformation, at each recursion, *DELISP* shrinks a data sequence by removing the matched prefix and the invalid postfix and finds elements incrementally by checking valid prefix subsequences only. The delimited growth technique further assures each growth satisfies the constraints and makes pattern-growth more efficient.

6. Conclusion

Generalized sequential patterns extend common patterns with time constraints, offer syntactic discovery for users, and broaden the applications. We have presented a new approach, *DELISP*, for the discovery of generalized sequential patterns. Incorporating the specified time constraints directly in the processes of projecting sub-sequences to sub-databases and growing frequent patterns, the pattern-growth based *DELISP* efficiently mines generalized sequential patterns. The conducted experiments confirm that with good linear scalability, *DELISP* outperforms the Apriori-based *GSP* algorithm.

Reference

[1] R. Agrawal and R. Srikant, "Mining Sequential Patterns," *Proceedings of the 11th International Conference on Data Engineering*, Taipei, Taiwan, pp. 3-14, 1995.

[2] C. Bettini, X. S. Wang, and S. Jajodia, "Mining Temporal Relationships with Multiple Granularities in Time Sequences," *Data Engineering Bulletin*, Vol. 21, pp. 32-38, 1998.

[3] M. N. Garofalakis, R. Rastogi, and K. Shim, "SPIRIT: Sequential Pattern Mining with Regular Expression Constraints," *Proceedings of the 25th International Conference on Very Large Data Bases*, Edinburgh, Scotland, pp. 223-234, 1999.

[4] J. Han, J. Pei, B. Mortazavi-Asl, Q. Chen, U. Dayal and M.-C. Hsu, "FreeSpan: Frequent pattern-projected sequential pattern mining," *Proceedings of the 6th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 355-359, 2000.

[5] M. Y. Lin and S. Y. Lee, "Incremental Update on Sequential Patterns in Large Databases," *Proceedings of 10th IEEE International Conference on Tools with Artificial Intelligence*, Taipei, Taiwan, pp. 24-31, 1998.

[6] H. Mannila, H. Toivonen and A. I. Verkamo, "Discovery of Frequent Episodes in Event Sequences," *Data Mining and Knowledge Discovery*, Vol. 1, Issue 3, pp. 259-289, 1997.

[7] F. Masseglia, F. Cathala, and P. Poncelet, "The PSP Approach for Mining Sequential Patterns," *Proceedings of 1998 2nd European Symposium on Principles of Data Mining and Knowledge Discovery*, Vol. 1510, Nantes, France, pp. 176-184, Sep. 1998.

[8] T. Oates, M. D. Schmill, D. Jensen, and P. R. Cohen, "A Family of Algorithms for Finding Temporal Structure in Data," *Proceedings of the 6th International Workshop on AI and Statistics*, Fort Lauderdale, Florida, pp. 371-378, 1997.

[9] J. Pei, J. Han, H. Pinto, Q. Chen, U. Dayal and M.-C. Hsu, "PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth," *Proceedings of 2001 International Conference on Data Engineering*, pp. 215-224, 2001.

[10] T. Shintani and M. Kitsuregawa, "Mining algorithms for sequential patterns in parallel: Hash based approach," *Proceedings of the Second Pacific–Asia Conference on Knowledge Discovery and Data mining*, pp. 283-294, 1998.

[11] R. Srikant and R. Agrawal, "Mining Sequential Patterns: Generalizations and Performance Improvements," *Proceedings* of the *5th International Conference on Extending Database Technology*, Avignon, France, pp. 3-17, 1996. (An extended version is the IBM Research Report RJ 9994)

[12] K. Wang, "Discovering patterns from large and dynamic sequential data," *Journal of Intelligent Information Systems*, Vol. 9, No. 1, pp. 33-56, 1997.

[13] M. J. Zaki, "Sequence Mining in Categorical Domains: Incorporating Constraints," *Proceedings of the 9th International Conference on Information and Knowledge Management*, Washington D.C., pp. 422-429, 2000.

[14] M. J. Zaki, "SPADE: An Efficient Algorithm for Mining Frequent Sequences," *Machine Learning Journal*, Vol. 42, No. 1/2, pp. 31-60, 2001.